Fayda API — Telegram Bot Integration

Purpose: Integration guide for Telegram bots that call the Fayda National ID Validation API and deliver the generated Fayda PDF to the user.

Base URL: `https://otp.affiliate.pro.et/api/v1`
Authentication: send `x-api-key: YOUR_KEY` on every API request.

1. Important API Response Change

The key change: `POST /verify-otp` does **not** return JSON when verification succeeds. A successful response is the actual PDF file as binary data.

Step	Success body	Content-Type
POST /send-otp	JSON: session + phone	application/json
POST /verify-otp	Raw PDF bytes	application/pdf
Any API error	JSON: success + message	application/json

Why the error happens

If the bot always calls `response.json()` or Axios uses its default JSON response handling for `/verify-otp`, the PDF begins with PDF data such as `%PDF-1.4`, which is not valid JSON. The result is an error such as **Verify-OTP endpoint returned invalid JSON**.

Correct approach: receive `/verify-otp` as binary and inspect `Content-Type`. Parse JSON only when the response is an API error.

2. Telegram Bot Flow

```
User sends FAN
  → bot POST /send-otp
  → bot stores session for that chat
  → bot tells user OTP was sent to masked phone

User sends OTP
  → bot POST /verify-otp with { otp, fanNumber, session }
  → if Content-Type is application/pdf:
        send PDF bytes to Telegram as a document
  → if Content-Type is application/json:
        show data.message to the user
```

State: store `session` and `fanNumber` per `chat_id`. Send the session back exactly as the API returned it. Do not decode, modify, trim, or reconstruct it.

3. API Request Details

3.1 Send OTP

```
POST /api/v1/send-otp
Content-Type: application/json
x-api-key: YOUR_KEY

{
  "fanNumber": "1234567890123456"
}
```

Success — HTTP 200:

```
{
  "session": "opaque-string-store-as-is",
  "phone": "******7890"
}
```

3.2 Verify OTP

```
POST /api/v1/verify-otp
Content-Type: application/json
x-api-key: YOUR_KEY
```

```
{
  "otp": "123456",
  "fanNumber": "1234567890123456",
  "session": "opaque-string-from-step-1"
}
```

Success — HTTP 200: body is the PDF file. Typical headers include `Content-Type: application/pdf`, a PDF filename in `Content-Disposition`, and `X-Remaining-Points`.

Error — HTTP 400 / 401 / 402 / 500:

```
{
  "success": false,
  "message": "Human-readable error"
}
```

4. Node.js — Telegraf + Axios

The following implementation explicitly requests `arraybuffer` for `/verify-otp`, detects PDF versus JSON, and sends the PDF to Telegram.

```
const { Telegraf, Input } = require('telegraf');
const axios = require('axios');

const BASE_URL = 'https://otp.affiliate.pro.et/api/v1';
const API_KEY = process.env.FAYDA_API_KEY;
const bot = new Telegraf(process.env.BOT_TOKEN);

const pending = new Map(); // chatId -> { fanNumber, session }

function faydaHeaders() {
  return {
    'Content-Type': 'application/json',
    'x-api-key': API_KEY,
  };
}

bot.command('start', (ctx) =>
  ctx.reply('Send a 16-digit FAN number to begin.')
);

bot.on('text', async (ctx) => {
  const chatId = ctx.chat.id;
  const text = ctx.message.text.trim();
  const state = pending.get(chatId);

  try {
    // User is sending OTP
    if (state?.session && /^\d{6}$/.test(text)) {
      await ctx.reply('Verifying OTP…');

      const res = await axios.post(
        `${BASE_URL}/verify-otp`,
        {
          otp: text,
          fanNumber: state.fanNumber,
          session: state.session,
        },
        {
          headers: faydaHeaders(),
          responseType: 'arraybuffer',
          validateStatus: () => true,
        }
      );

      const contentType = String(res.headers['content-type'] || '');

      if (contentType.includes('application/pdf') && res.status === 200) {
        const filename =
          /filename="([^"]+)"/.exec(
            res.headers['content-disposition'] || ''
          )?.[1] || `Fayda-${state.fanNumber}.pdf`;

        pending.delete(chatId);

        await ctx.replyWithDocument(
          Input.fromBuffer(Buffer.from(res.data), filename)
        );
        return;
      }

      // Error body is JSON, but arraybuffer was requested.
      let message = 'Verification failed.';
```

```
      try {
        const err = JSON.parse(Buffer.from(res.data).toString('utf8'));
        message = err.message || message;
      } catch {
        // Non-JSON fallback
        message = `Verification failed (HTTP ${res.status}).`;
      }

      pending.delete(chatId);
      await ctx.reply(message);
      return;
    }

    // User is sending FAN
    const fanNumber = text.replace(/\s+/g, '');

    if (!/^\d{16}$/.test(fanNumber)) {
      await ctx.reply(
        'Send a 16-digit FAN, or a 6-digit OTP after requesting one.'
      );
      return;
    }

    const step1 = await axios.post(
      `${BASE_URL}/send-otp`,
      { fanNumber },
      { headers: faydaHeaders(), timeout: 60000 }
    );

    pending.set(chatId, {
      fanNumber,
      session: step1.data.session,
    });

    await ctx.reply(
      `OTP sent to ${step1.data.phone}. Reply with the 6-digit code.`
    );
  } catch (err) {
    let message = 'Request failed.';

    if (err.response?.data) {
      if (Buffer.isBuffer(err.response.data)) {
        try {
          const parsed = JSON.parse(
            err.response.data.toString('utf8')
          );
          message = parsed.message || message;
        } catch {}
      } else {
        message = err.response.data.message || message;
      }
    } else if (err.message) {
      message = err.message;
    }

    await ctx.reply(String(message));
  }
});

bot.launch();
```

Critical Axios setting

`responseType: 'arraybuffer'` is required for `/verify-otp`. Without it, Axios may attempt to parse the PDF as JSON and produce the invalid-JSON error.

5. Python — python-telegram-bot + requests

This version reads the verify response as raw bytes through `res.content`, checks the response Content-Type, and sends the PDF as a Telegram document.

```
import io
import os
import re
import requests
from telegram import Update
from telegram.ext import Application, MessageHandler, filters, ContextTypes

BASE_URL = "https://otp.affiliate.pro.et/api/v1"
API_KEY = os.environ["FAYDA_API_KEY"]
HEADERS = {
    "x-api-key": API_KEY,
```

```
    "Content-Type": "application/json",
}

pending = {}  # chat_id -> {fanNumber, session}


async def handle_text(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    chat_id = update.effective_chat.id
    text = (update.message.text or "").strip()
    state = pending.get(chat_id)

    # OTP
    if state and re.fullmatch(r"\d{6}", text):
        await update.message.reply_text("Verifying OTP…")

        res = requests.post(
            f"{BASE_URL}/verify-otp",
            json={
                "otp": text,
                "fanNumber": state["fanNumber"],
                "session": state["session"],
            },
            headers=HEADERS,
            timeout=120,
        )

        content_type = res.headers.get("Content-Type", "")

        if res.status_code == 200 and "application/pdf" in content_type:
            filename = "fayda.pdf"

            disp = res.headers.get("Content-Disposition", "")
            match = re.search(r'filename="([^"]+)"', disp)

            if match:
                filename = match.group(1)

            pending.pop(chat_id, None)

            await update.message.reply_document(
                document=io.BytesIO(res.content),
                filename=filename,
            )
            return

        # Error is normally JSON.
        if "application/json" in content_type:
            try:
                err = res.json()
            except ValueError:
                err = {"message": "Invalid JSON error response."}
        else:
            err = {"message": res.text}

        pending.pop(chat_id, None)

        await update.message.reply_text(
            err.get("message", "Verification failed.")
        )
        return

    # FAN
    fan = re.sub(r"\s+", "", text)

    if not re.fullmatch(r"\d{16}", fan):
        await update.message.reply_text(
            "Send a 16-digit FAN, or a 6-digit OTP after requesting one."
        )
        return

    step1 = requests.post(
        f"{BASE_URL}/send-otp",
        json={"fanNumber": fan},
        headers=HEADERS,
        timeout=60,
    )
    step1.raise_for_status()

    data = step1.json()

    pending[chat_id] = {
        "fanNumber": fan,
```

```
        "session": data["session"],
    }

    await update.message.reply_text(
        f"OTP sent to {data['phone']}. "
        "Reply with the 6-digit code."
    )


app = Application.builder().token(
    os.environ["BOT_TOKEN"]
).build()

app.add_handler(
    MessageHandler(
        filters.TEXT & ~filters.COMMAND,
        handle_text
    )
)

app.run_polling()
```

6. Production Checklist

- ■ Stop parsing `/verify-otp` as JSON on successful verification.
- ■ Use binary response handling: Axios `arraybuffer` or Python `res.content`.
- ■ Branch on `Content-Type`: PDF → send as a Telegram document; JSON → show the API `message`.
- ■ Keep `session` and `fanNumber` associated with the correct `chat_id` and send the session back unchanged.
- ■ Use approximately 60 seconds for `send-otp` and 120 seconds for `verify-otp`, or adjust according to your deployment.
- ■ Keep the Fayda API key on the bot server. Never expose it in a Telegram Mini App, frontend, browser, or other client-side code.
- ■ For multiple bot instances, replace an in-memory `Map`/`dict` with shared storage such as Redis or a database so the session is available regardless of which instance receives the OTP.

7. Common Errors

Message / status	Meaning / fix
Verify-OTP endpoint returned invalid JSON	Bot expected JSON, but successful verify-otp returned a PDF. Use binary response handling.
401 — Invalid API key	The x-api-key is missing, incorrect, expired, or otherwise rejected.
402 — Insufficient points	The API account does not have enough points; recharge the account.
400 — otp / fanNumber / session	A required field is missing or malformed.
session is invalid or malformed	The session was changed, truncated, or does not belong to the corresponding send-otp request.
Timeout	Increase the client timeout. PDF generation can take several seconds.

8. Recommended Response-Handling Rule

```
if Content-Type contains "application/pdf" AND status == 200:
    treat response body as binary PDF
    send bytes to Telegram as document
else if Content-Type contains "application/json":
    parse JSON
    show the "message" field
else:
    treat as unexpected response
    log status + content type
    do not blindly call response.json()
```

Security Note

The `session` is an opaque value and should be treated as sensitive request state. Do not expose it in logs, Telegram messages, frontend code, or public URLs. Also avoid logging the full FAN and OTP.

Summary

The integration issue is caused by a response-format mismatch: `send-otp` returns JSON, while a successful `verify-otp` returns a PDF. The bot must therefore request and handle the verification response as binary data, then use the Content-Type to distinguish a PDF success response from a JSON error response.